

```
int main( )
{
    char str[ ] = "Open Source Software is Good";
    regex pat("Good");
    char rep[ ] = "Excellent";
    cout <<regex_replace(str,pat,rep)<<'\\n';
    return 0;
}
```

On execution with the commands `g++ regex4.cc` and `./a.out`, the program `regex4.cc` prints the message *Open Source Software is Excellent*. This clearly tells us that the function `regex_replace()` behaves like the function `regex_search()` whereby the whole string is searched for a possible match for the given regular expression, unlike the function `regex_match()` which looks for a match at the very beginning of a string. Figure 2 shows the output of the two programs, `regex3.cc` and `regex4.cc`.

File processing in C++ with regular expressions

The next question that needs to be answered is: How do we process data inside a text file with a regular expression? To test the working of such a program, a text file called `file1.txt` is used, which is the same one used in the previous articles in this series on regular expressions.

```
unix is an operting system
Unix is an Operating System
UNIX IS AN OPERATING SYSTEM
Linux is also an Operating System
```

Now let us consider the following C++ program called `regex5.cc` that reads and processes the text file `file1.txt` line by line, to print all the lines that contain the word 'UNIX'.

```
#include <iostream>
#include <string>
#include <fstream>
#include <regex>

using namespace std;

int main( )
{
    ifstream file("file1.txt");
    string str;
    regex pat("UNIX");
    while (getline(file, str))
    {
        if( regex_search(str,pat) )
        {
            cout << str <<"\\n";
        }
    }
}
```

```
return 0;
}
```

When the program `regex5.cc` is executed with the commands `g++ regex5.cc` and `./a.out`, the message printed on the screen is *UNIX IS AN OPERATING SYSTEM*. So, a case-sensitive pattern match is carried out here. The next question is: How do we carry out a case-insensitive pattern match? For this purpose, we use a regex constant called `icase`. When the line of code `regex pat("UNIX");` is replaced with the line `regex pat("UNIX", regex_constants::icase);` a case insensitive match is carried out, and this results in a match for three lines in the text file `file1.txt`. Figure 3 shows the results of the case-sensitive and case-insensitive matches. There are many other regex constants defined in the namespace `regex_constants`. Some of them are `nosubs`, `optimize`, `collate`, etc. Use of these regex constants will add more power to your regular expressions. It is a good idea to learn more about them as you gain more information about C++ regular expressions.

```
# g++ regex3.cc
# ./a.out
Free Software is Good
# g++ regex4.cc
# ./a.out
Open Source Software is Excellent
#
```

Figure 2: The function `regex_replace()` in C++

```
# g++ regex5.cc
# ./a.out
UNIX IS AN OPERATING SYSTEM
# g++ regex6.cc
# ./a.out
unix is an operting system
Unix is an Operating System
UNIX IS AN OPERATING SYSTEM
#
```

Figure 3: Case-sensitive and case-insensitive matches

Regular expressions in C++14 and C++17

It is now time for us to discuss regular expressions in C++14. Luckily, except for a few minor additions, the `<regex>` header file of C++11 has remained largely unchanged even after the introduction of the later standard C++14. For example, the definitions of the functions `regex_match()` and `regex_search()` are slightly modified in C++14 so that additional processing with these functions is possible. But these changes only add more power to existing functions and do not affect their basic working. And finally, what are the changes that will be brought on by C++17? Hopefully, nothing major. So far, there have been no rumours about whether there will be a major revision to the header file `<regex>`. Therefore, whatever we have learnt from this article can be used for a long time.

Regular expression style of C++

Unlike the previous two articles in this series, in this article I have started by explaining C++ code snippets using regular expressions directly, without providing details regarding the